

Create authentication service that returns JWT

pom.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.3</version>
    <relativePath/>
  </parent>

  <groupId>com.cognizant</groupId>
  <artifactId>spring-learn</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>spring-learn</name>

  <properties>
    <java.version>17</java.version>
  </properties>

  <dependencies>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-security</artifactId>
    </dependency>

    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context</artifactId>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-devtools</artifactId>
      <scope>runtime</scope>
      <optional>true</optional>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>

    <!-- JWT (modern API, no javax.xml.bind dependency) -->
    <dependency>
      <groupId>io.jsonwebtoken</groupId>
      <artifactId>jjwt-api</artifactId>
      <version>0.11.5</version>
    </dependency>
  </dependencies>
```

```

        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-impl</artifactId>
        <version>0.11.5</version>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-jackson</artifactId>
        <version>0.11.5</version>
        <scope>runtime</scope>
    </dependency>
</dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>

```

application.properties:

```

spring.application.name=spring-learn
server.port=8090
logging.level.root=INFO
logging.level.com.cognizant.spring_learn=DEBUG

```

SecurityConfig.java:

```

package com.cognizant.spring_learn.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;

@Configuration
public class SecurityConfig {

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http)
        throws Exception {

        http
            .authorizeHttpRequests(auth -> auth

                .requestMatchers("/authenticate")
                .hasAnyRole("USER", "ADMIN")

                .requestMatchers("/countries")
                .hasRole("USER")
            )
    }
}

```

```

        .anyRequest()
        .authenticated()

    )
    .httpBasic(Customizer.withDefaults())
    .csrf(csrf -> csrf.disable());

    return http.build();
}

@Bean
public UserDetailsService users() {

    UserDetails user =
        User.withUsername("user")
            .password("{noop}pwd")
            .roles("USER")
            .build();

    UserDetails admin =
        User.withUsername("admin")
            .password("{noop}pwd")
            .roles("ADMIN")
            .build();

    return new InMemoryUserDetailsManager(
        user,
        admin);
}
}

```

AuthenticationController.java:

```

package com.cognizant.spring_learn.controller;

import java.nio.charset.StandardCharsets;
import java.util.Base64;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestHeader;
import org.springframework.web.bind.annotation.RestController;

import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.JwtBuilder;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.security.Keys;

import java.security.Key;

@RestController
public class AuthenticationController {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(AuthenticationController.class);

    // Must be at least 256 bits (32 bytes) for HS256
    private static final Key SECRET_KEY =
        Keys.hmacShaKeyFor(
            "secretkeysecretkeysecretkey123456".getBytes(StandardCharsets.UTF_8));

    @GetMapping("/authenticate")

```

```
public Map<String, String> authenticate(
    @RequestHeader("Authorization") String authHeader) {

    LOGGER.debug("START");

    LOGGER.debug("Authorization Header : {}", authHeader);

    String user = getUser(authHeader);

    String token = generateJwt(user);

    Map<String, String> map = new HashMap<>();

    map.put("token", token);

    LOGGER.debug("END");

    return map;
}

private String getUser(String authHeader) {

    LOGGER.debug("START getUser()");

    String encodedCredentials =
        authHeader.substring("Basic ".length());

    byte[] decodedBytes =
        Base64.getDecoder().decode(encodedCredentials);

    String credentials = new String(decodedBytes, StandardCharsets.UTF_8);

    LOGGER.debug("Decoded Credentials : {}", credentials);

    String user = credentials.split(":")[0];

    LOGGER.debug("User : {}", user);

    LOGGER.debug("END getUser()");

    return user;
}

private String generateJwt(String user) {

    LOGGER.debug("START generateJwt()");

    JwtBuilder builder = Jwts.builder();

    builder.setSubject(user);

    builder.setIssuedAt(new Date());

    builder.setExpiration(
        new Date(System.currentTimeMillis() + 1200000));

    builder.signWith(SECRET_KEY, SignatureAlgorithm.HS256);

    String token = builder.compact();

    LOGGER.debug("Generated Token : {}", token);

    LOGGER.debug("END generateJwt()");

    return token;
}
```

```
}  
}
```

SpringLearnApplication.java:

```
package com.cognizant.spring_learn;
```

```
import org.springframework.boot.SpringApplication;
```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
@SpringBootApplication
```

```
public class SpringLearnApplication {
```

```
    public static void main(String[] args) {
```

```
        SpringApplication.run(SpringLearnApplication.class, args);
```

```
    }
```

```
}
```

OUTPUT:

